K. J. Somaiya School of Engineering, Mumbai-77

(Somaiya Vidyavihar University)

Department of Computer Engineering

Course Name:	Web Development Laboratory (316U01L306)	Semester:	III
Date of Performance:	_23__ / __09_ / _2026_____	DIV/ Batch No:	B-2
Student Name:	Ishaan Singh Khanka	Roll No:	16010125107

Experiment No: 4

Title

Form Validation using JavaScript for a Climate Intelligence and Heatwave Early Warning Web Portal

Aim of the Experiment

To design and develop a JavaScript-based form validation module for a Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning web portal, ensuring that user-entered weather monitoring, stakeholder, and alert-related information is complete, valid, and correctly formatted.

Objectives of the Experiment

- To create an interactive HTML form for collecting heatwave monitoring and alert-related information.
- To validate form fields using JavaScript event handling and conditional statements.
- To apply Regular Expressions for validating name, mobile number, email address, and AWS station ID.
- To display meaningful error messages for invalid inputs and confirmation messages for valid inputs.
- To demonstrate client-side validation as part of a climate intelligence decision-support platform.

COs to be achieved

CO1: Developing webpages using HTML, CSS and JavaScript.

Books / Journals / Websites references

1. MDN Web Docs – JavaScript and HTML Form Validation

2. W3schools.com

Theory

Form validation is the process of checking whether data entered by a user satisfies predefined rules before it is submitted. JavaScript provides client-side validation using events, conditional statements, functions, string

methods, and Regular Expressions. It gives immediate feedback and helps prevent incomplete or incorrectly formatted data.

In this experiment, validation is applied to a Climate Intelligence and Heatwave Early Warning web portal. The use case combines meteorological observations, IoT-enabled Automated Weather Stations (AWS), AI-driven heatwave analytics, visualization, and advisory mechanisms. The form collects station, location, temperature, humidity, date, observer, and alert information.

Problem Statement

"Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning"

Develop a JavaScript-enabled web form for collecting localized weather monitoring information and stakeholder details for a heatwave monitoring and early warning platform. The form should validate mandatory fields and ensure that temperature, humidity, email, mobile number, AWS station ID, and observation date are entered in the correct format.

Task 1: Create the Heatwave Monitoring Form

Scenario

Design a web form for entering localized heatwave monitoring information.

- Observer Name, Email, Mobile Number, AWS Station ID, Location, Observation Date, Maximum Temperature, Humidity, and Heatwave Alert Level.
- Use suitable HTML5 input types such as text, email, tel, date and number.
- Use labels, fieldset and legend.
- Mark essential fields as required.

Concepts Tested

- HTML Forms, HTML5 Input Types, Form Controls, Labels and Fieldset

Deliverable: heatwave-form.html

Task 2: Validate User Name and Station ID

Scenario

Ensure that the observer name and AWS Station ID contain valid characters and are not blank.

Requirements

- Name must contain alphabets and spaces only and have at least 3 characters.
- AWS Station ID should follow a format such as AWS001.
- Display an error message for invalid values.

Concepts Tested

- JavaScript Functions, Conditional Statements, Regular Expressions, String Validation

Task 3: Validate Email and Mobile Number

Scenario

Validate contact information before accepting the monitoring form.

Requirements

- Validate email using a Regular Expression.
- Validate a 10-digit Indian mobile number.
- Display field-level error messages.

Concepts Tested

- Regular Expressions, JavaScript String Methods, Error Messages

Task 4: Validate Weather Observation Data

Scenario

Ensure temperature and humidity observations are within reasonable input ranges.

Requirements

- Maximum Temperature must be numeric.
- Humidity must be between 0 and 100 percent.
- Observation date must not be empty.
- Display messages for missing or out-of-range values.

Concepts Tested

- Number Validation, Range Checking, Date Validation, JavaScript Conditions

Task 5: Validate Alert Level and Submit Form

Scenario

Validate the Inputs and display a success message when all inputs are correct.

Requirements

- Display a success message only when all rules are satisfied.

Concepts Tested

- Form Events, submit Event, preventDefault(), DOM Manipulation, Client-side Validation

Deliverable: heatwave-validation.html

Code



```
<script>
    // Expt 4: JavaScript form validation script
    // Each validation function checks a specific field and shows custom error messages.

    const form = document.getElementById('heatwaveForm');
    const successMessage = document.getElementById('formSuccess');

    function showError(fieldId, message) {
        const errorField = document.getElementById(fieldId + 'Error');
        errorField.textContent = message;
    }

    function clearError(fieldId) {
        showError(fieldId, '');
    }

    // Task 2: Name and station ID validation
    function validateObserverName() {
        const value = document.getElementById('observerName').value.trim();
        if (value === '') {
            showError('observerName', 'Observer name is required.');
            return false;
        }
        if (!/^[A-Za-z ]{3,}$/.test(value)) {
            showError('observerName', 'Name must contain alphabets and spaces only, with at least 3 characters.'
            return false;
        }
        clearError('observerName');
        return true;
    }
```

```
    function validateAwsStationId() {
        const value = document.getElementById('awsStationId').value.trim();
        if (value === '') {
            showError('awsStationId', 'AWS Station ID is required.');
            return false;
        }
        if (!/^AWS\d{3,}$/i.test(value)) {
            showError('awsStationId', 'AWS Station ID must follow the format AWS001.');
            return false;
        }
        clearError('awsStationId');
        return true;
    }

    // Task 3: Email and mobile validation using Regular Expressions
    function validateEmail() {
        const value = document.getElementById('email').value.trim();
        const emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;

        if (value === '') {
            showError('email', 'Email is required.');
            return false;
        }
        if (!emailPattern.test(value)) {
            showError('email', 'Please enter a valid email address.');
            return false;
        }
        clearError('email');
        return true;
    }
```

```
<script>
    function validateMobile() {
        const value = document.getElementById('mobile').value.trim();
        if (value === '') {
            showError('mobile', 'Mobile number is required.');
            return false;
        }
        if (!/^\d{10}$/.test(value)) {
            showError('mobile', 'Mobile number must contain exactly 10 digits.');
            return false;
        }
        clearError('mobile');
        return true;
    }

    // Task 4: Validate weather observation data and ranges
    function validateLocation() {
        const value = document.getElementById('location').value.trim();
        if (value === '') {
            showError('location', 'Location is required.');
            return false;
        }
        clearError('location');
        return true;
    }

    function validateObservationDate() {
        const value = document.getElementById('observationDate').value;
        if (!value) {
            showError('observationDate', 'Observation date is required.');
            return false;
```

```
            return false;
        }
        clearError('observationDate');
        return true;
    }

    function validateTemperature() {
        const value = Number(document.getElementById('temperature').value);
        if (document.getElementById('temperature').value.trim() === '') {
            showError('temperature', 'Temperature is required.');
            return false;
        }
        if (isNaN(value) || value < 0 || value > 60) {
            showError('temperature', 'Temperature should be between 0°C and 60°C.');
            return false;
        }
        clearError('temperature');
        return true;
    }

    function validateHumidity() {
        const value = Number(document.getElementById('humidity').value);
        if (document.getElementById('humidity').value.trim() === '') {
            showError('humidity', 'Humidity is required.');
            return false;
        }
        if (isNaN(value) || value < 0 || value > 100) {
```

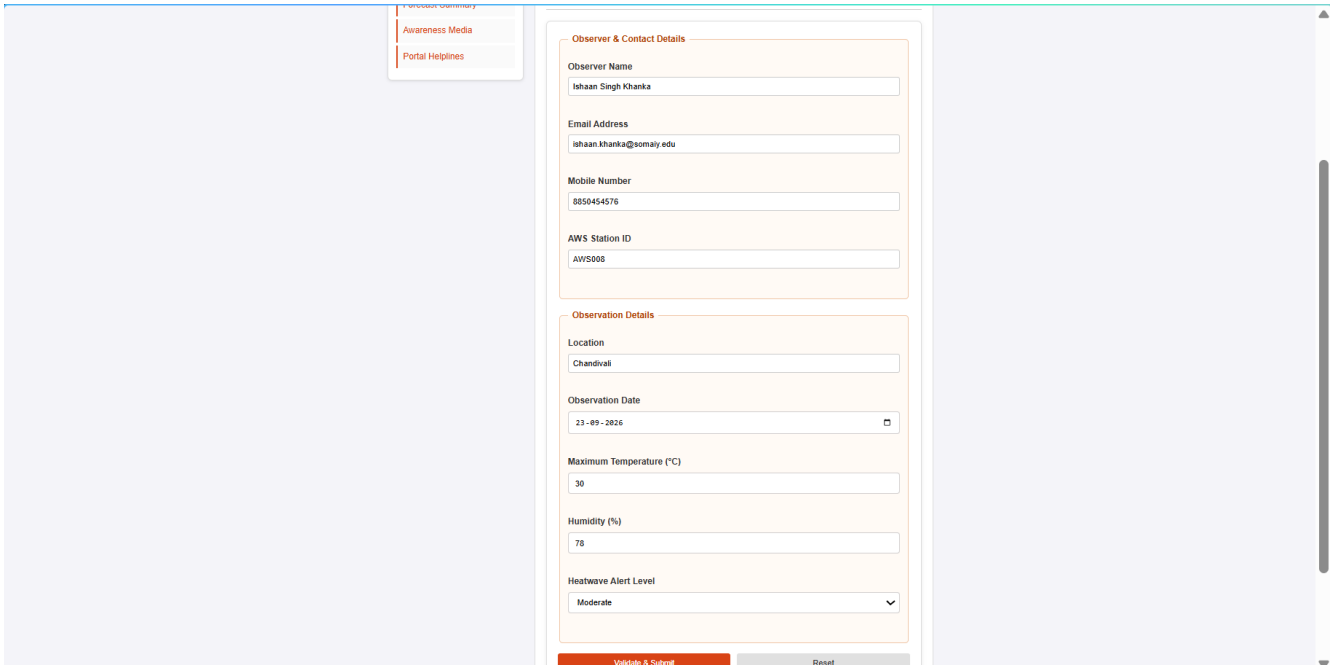
```
        if (isNaN(value) || value < 0 || value > 100) {
            showError('humidity', 'Humidity must be between 0% and 100%.');
            return false;
        }
        clearError('humidity');
        return true;
    }

    function validateAlertLevel() {
        const value = document.getElementById('alertLevel').value;
        if (value === '') {
            showError('alertLevel', 'Please select a heatwave alert level.');
            return false;
        }
        clearError('alertLevel');
        return true;
    }

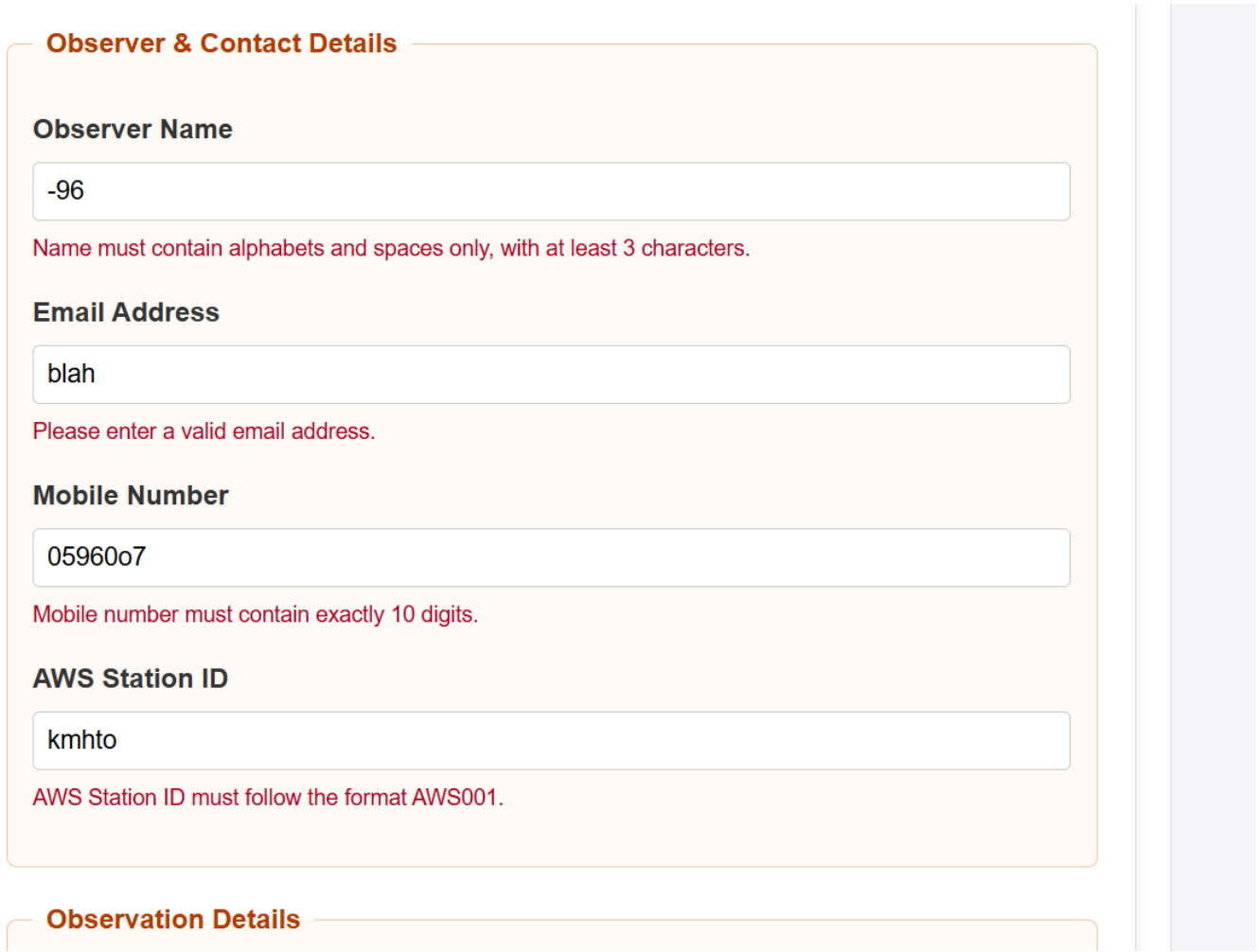
    // Task 5: Final validation on form submission
    function validateForm() {
        const results = [
            validateObserverName(),
            validateAwsStationId(),
            validateEmail(),
            validateMobile(),
            validateLocation(),
            validateObservationDate(),
            validateTemperature()
```

Output / Screenshot

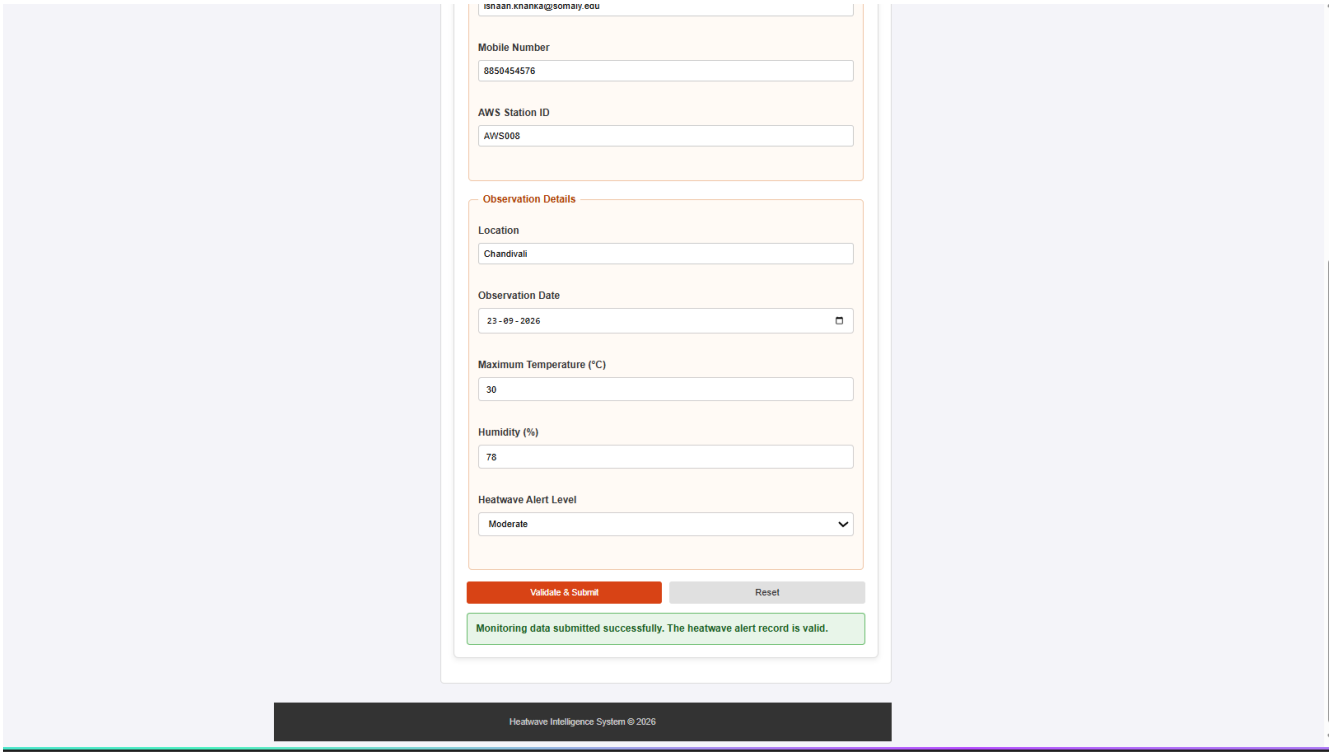
Students should execute the HTML file in a web browser and capture screenshots showing:

- The completed Heatwave Monitoring form.
- 

- Validation error messages for invalid inputs.



- Successful submission message for valid inputs.



Conclusion and Discussion

The experiment demonstrates how JavaScript can be used for client-side form validation in a web-based Climate Intelligence system. Validation rules ensure that monitoring and stakeholder information is complete and correctly formatted before processing. The experiment demonstrates JavaScript functions, events, conditional statements, Regular Expressions, DOM manipulation, and preventDefault(). The module can serve as a front-end component of a larger heatwave monitoring, visualization, advisory, and early warning platform.